

AIQT CLI

AIQT Technical Architecture Specification v0.3

Lean Workflow Engine Architecture

Field	Value
Product	AIQT CLI
Specification	Technical Architecture Specification
Version	v0.3
Aligns With	AIQT MVP Specification v0.6
Status	Implementation-aligned architecture
Architecture Positioning	Local CLI workflow state engine
Primary Runtime	Node.js + TypeScript CLI

1. Purpose

This document defines the technical architecture for building AIQT as a Node.js/TypeScript CLI aligned with the AIQT MVP Specification v0.6. The product specification defines what AIQT should do. This architecture defines how AIQT should be structured technically.

AIQT must behave as a local workflow state engine that converts a rough project idea into structured state, generates a dynamic work graph, creates bounded agent handoff packets, captures checkpoints, and continues from saved state.

2. v0.3 Architecture Change Summary

- Defines all previously referenced schemas: AssumptionSchema, QualityExpectationsSchema, and IntegrationsSchema.
- Clarifies that canonical JSON files use camelCase.
- Introduces AIQT_SCHEMA_VERSION as the file schema version, distinct from package.json.version.
- Defines exit code 3 for unsupported future schema versions, missing versions, invalid versions, and malformed canonical JSON.
- Clarifies that Milestone 1 init is non-interactive. Interactive intake is deferred to aiqt update.
- Defines runlog health behavior for malformed JSONL lines.

3. Technical Goals

- Provide a TypeScript CLI executable named aiqt.
- Use a local file-based state engine.

- Use a minimal canonical file model.
- Store structured project context in project.json.
- Store workflow state and dynamic work graph in state.json.
- Store append-only workflow history in runlog.jsonl.
- Expose a small public command surface.
- Support human-readable and deterministic JSON output modes.
- Support a resumable workflow loop.
- Generate bounded agent handoff packets.
- Capture checkpoints and update work-unit status.
- Validate all canonical state files with schemas.
- Use safe writes, atomic JSON writes, and predictable exit codes.

4. Technical Non-Goals

- Web dashboard or hosted backend.
- Multi-user collaboration or cloud synchronization.
- Real-time agent orchestration.
- Direct Claude, Codex, or OpenAI API calls.
- Automatic code modification.
- Full issue tracker replacement.
- Graph database or workflow DSL.
- Persistent markdown files for every milestone, phase, step, or task.
- Required human-readable documents during normal workflow.

5. Runtime and Libraries

Area	Decision
Runtime	Node.js latest active LTS
Package manager	pnpm
Module format	ESM
Language	TypeScript
CLI framework	commander
Interactive prompts	@inquirer/prompts, deferred outside M1 except where explicitly scoped
Schema validation	zod
Testing	vitest
Terminal output	chalk, optional
Clipboard	clipboardy, optional for aiqt next --copy

Use minimal dependencies. Do not add a library until it solves a concrete MVP need. Do not introduce a graph database or use markdown as a source-of-truth dependency.

6. Canonical File Model

```
.aiqt/
  project.json
  state.json
```

```
runlog.jsonl
exports/
```

File	Purpose
project.json	Durable project context and durable project knowledge.
state.json	Workflow position, work graph, checkpoints, current work unit, next command.
runlog.jsonl	Append-only event history, one valid JSON object per line.
exports/	Optional generated markdown views only.

6.1 File Creation Rules

aiqt init creates only the canonical files and exports directory. It must not create AIQT.md, AGENTS.md, CLAUDE.md, docs/, .milestones/, .tasks/, .aiqt/session.json, .aiqt/validation-history.json, or .aiqt/agent-actions.log.

7. Repository Structure

```
aiqt/
  package.json
  pnpm-lock.yaml
  tsconfig.json
  README.md
  src/
    index.ts
    cli/
      register-commands.ts
      command-context.ts
      options.ts
      commands/
        init.command.ts
        update.command.ts
        plan.command.ts
        next.command.ts
        checkpoint.command.ts
        review.command.ts
        export.command.ts
        status.command.ts
    core/
      constants/
        schema-version.ts
      output/
        human-output.ts
        json-output.ts
        exit-codes.ts
        result.ts
        issue.ts
      filesystem/
        paths.ts
        file-store.ts
        safe-writer.ts
        atomic-write.ts
        file-exists.ts
      schema/
        project.schema.ts
        state.schema.ts
        work-graph.schema.ts
        milestone.schema.ts
        work-unit.schema.ts
        dependency.schema.ts
        decision.schema.ts
        checkpoint.schema.ts
        runlog-event.schema.ts
        common.schema.ts
```

```

state/
  project-store.ts
  workflow-state-store.ts
  runlog-store.ts
  versioning.ts
  ids.ts
workflow/
  workflow-engine.ts
  actions.ts
  statuses.ts
  next-action.ts
  gates.ts
  transitions.ts
services/
  project-update-service.ts
  planning-service.ts
  agent-packet-service.ts
  checkpoint-service.ts
  review-service.ts
  export-service.ts
templates/
  agent-packet.md.template
exports/
  project-plan.md.template
  status-report.md.template
  technical-spec.md.template
  agent-packet.md.template
tests/
  unit/
  integration/
  fixtures/

```

Milestone-specific build specs may implement a subset of this tree. Milestone 1 intentionally excludes planning, checkpoint, review, export, and agent-packet services.

8. Public Command Surface

Command	Intent	Writes State
aiqt init	Initialize canonical files	Yes
aiqt update	Update durable project context	Yes
aiqt plan	Generate or refresh work graph	Yes
aiqt next	Select next work unit and generate packet	Yes after full implementation
aiqt checkpoint	Capture agent execution result	Yes
aiqt review	Inspect quality/blockers and suggest corrections	Usually no
aiqt export	Generate human-readable output	Exports only
aiqt status	Read current workflow state	No

Commands returning meaningful workflow state must support `--json`. Commands writing generated files must support `--dry-run` where applicable.

9. Version Constants and Compatibility

```

// src/core/constants/schema-version.ts
export const AIQT_SCHEMA_VERSION = "0.5.0" as const;

// package.json.version is the npm package version.
// project.json.version and state.json.version are schema versions.
// Compare canonical file versions against AIQT_SCHEMA_VERSION, not package.json.version.

```

File Version Case	Behavior	Exit Code
Current supported version	Proceed	0
Older compatible version	Proceed only if schema validates	0 or 3
Unsupported future version	Block with upgrade message	3
Missing version	Block	3
Invalid version type	Block	3
Malformed project.json or state.json	Block	3

Unsupported future version output should say: AIQT cannot continue. Reason: This project was created with a newer AIQT version. Suggested action: Upgrade AIQT to the latest version.

10. Exit Codes

```
export const ExitCode = {
  Success: 0,
  ValidationFailed: 1,
  WorkflowBlocked: 2,
  InvalidInput: 3,
  MissingDependency: 4,
  ExternalIntegrationError: 5,
  HumanInputRequired: 10,
} as const;
```

Exit Code	Meaning
0	Success
1	Validation or review failed
2	Workflow action blocked
3	Invalid command, invalid input, invalid state, malformed canonical JSON, or unsupported schema version
4	Missing dependency required for requested action
5	External integration error
10	Human input required

11. Command Result Contract

```
export type CommandStatus =
  | "passed"
  | "failed"
  | "blocked"
  | "warning"
  | "needs_input";

export type WorkflowAction =
  | "init"
  | "update"
  | "plan"
  | "next"
  | "checkpoint"
  | "review"
  | "export"
  | "status";

export interface CommandResult<TData = unknown> {
```

```

status: CommandStatus;
action: WorkflowAction;
projectStatus: ProjectStatus | null;
currentMilestoneId: string | null;
currentWorkUnitId: string | null;
summary: string;
completedActions: string[];
changedFiles: string[];
affectedItems: string[];
blockingIssues: Issue[];
warnings: Issue[];
requiresHumanInput: boolean;
nextRecommendedCommand: string | null;
data?: TData;
exitCode: number;
}

```

12. Issue Contract

```

export interface Issue {
  id: string;
  severity: "low" | "medium" | "high" | "critical";
  area: string;
  message: string;
  affectedItems?: string[];
  suggestedAction?: string;
  agentCanFix?: boolean;
}

```

13. Schema Contracts

13.1 Common Rules

- Canonical JSON uses camelCase field names.
- ISO_DATE means an ISO 8601 timestamp string.
- IDs are stable after creation and must not be regenerated during export.
- Deleting an item must not cause existing IDs to be renumbered.

13.2 Project Model Schema

```

export const ProjectModelSchema = z.object({
  version: z.string(),
  project: ProjectSchema,
  context: ProjectContextSchema,
  requirements: z.array(RequirementSchema),
  decisions: z.array(DecisionSchema),
  risks: z.array(RiskSchema),
  assumptions: z.array(AssumptionSchema),
  openQuestions: z.array(OpenQuestionSchema),
  quality: QualityExpectationsSchema,
  integrations: IntegrationsSchema.optional(),
});

```

13.3 Project Schema

```

export const ProjectSchema = z.object({
  id: z.string(),
  name: z.string(),
  objective: z.string(),
  targetUsers: z.array(z.string()),
  preferredAgent: z.string().nullable(),
  existingRepositoryPath: z.string().nullable(),
});

```

```

    createdAt: z.string(),
    updatedAt: z.string(),
  });

```

13.4 Project Context Schema

```

export const ProjectContextSchema = z.object({
  constraints: z.array(z.string()),
  nonGoals: z.array(z.string()),
  technologyPreferences: z.array(z.string()),
  businessRules: z.array(z.string()),
  architectureNotes: z.array(z.string()),
});

```

13.5 Requirement Schema

```

export const RequirementSchema = z.object({
  id: z.string(),
  title: z.string(),
  description: z.string(),
  priority: z.enum(["low", "medium", "high", "critical"]),
  type: z.enum(["functional", "non_functional", "business", "security", "ux", "technical"]),
  acceptanceCriteria: z.array(z.string()),
  status: z.enum(["draft", "accepted", "implemented", "validated", "deferred"]),
});

```

13.6 Decision, Risk, Open Question

```

export const DecisionSchema = z.object({
  id: z.string(),
  date: z.string(),
  decision: z.string(),
  reason: z.string(),
  impact: z.string(),
  status: z.enum(["proposed", "decided", "reversed", "deferred"]),
});

export const RiskSchema = z.object({
  id: z.string(),
  title: z.string(),
  description: z.string(),
  severity: z.enum(["low", "medium", "high", "critical"]),
  mitigation: z.string().nullable(),
  status: z.enum(["open", "mitigated", "accepted", "closed"]),
});

export const OpenQuestionSchema = z.object({
  id: z.string(),
  question: z.string(),
  impact: z.enum(["low", "medium", "high", "blocking"]),
  status: z.enum(["open", "answered", "deferred"]),
  answer: z.string().nullable(),
});

```

13.7 Assumption Schema

```

export const AssumptionSchema = z.object({
  id: z.string(),
  statement: z.string(),
  reason: z.string().nullable(),
  source: z.enum(["human", "aiqt", "agent", "system"]),
  status: z.enum(["active", "replaced", "invalidated"]),
  createdAt: z.string(),
  updatedAt: z.string(),
});

```

13.8 Quality Expectations and Integrations

```
export const QualityExpectationsSchema = z.object({
  acceptanceCriteriaRequired: z.boolean(),
  validationRequiredBeforeDone: z.boolean(),
  preferredValidationCommands: z.array(z.string()),
});

export const IntegrationsSchema = z.record(z.unknown());
```

13.9 Workflow State Schema

```
export const WorkflowStateSchema = z.object({
  version: z.string(),
  projectStatus: z.enum(ProjectStatuses),
  currentMilestoneId: z.string().nullable(),
  currentWorkUnitId: z.string().nullable(),
  workGraph: WorkGraphSchema,
  checkpoints: z.array(CheckpointSchema),
  lastAgentPacket: AgentPacketMetadataSchema.nullable(),
  nextRecommendedCommand: z.string().nullable(),
  lastUpdatedAt: z.string(),
});
```

13.10 Work Graph Schemas

```
export const WorkGraphSchema = z.object({
  milestones: z.array(MilestoneSchema),
  workUnits: z.array(WorkUnitSchema),
  dependencies: z.array(DependencySchema),
});

export const MilestoneSchema = z.object({
  id: z.string(),
  title: z.string(),
  objective: z.string(),
  status: z.enum(["planned", "ready", "in_progress", "blocked", "done", "cancelled"]),
  workUnitIds: z.array(z.string()),
});

export const WorkUnitSchema = z.object({
  id: z.string(),
  milestoneId: z.string().nullable(),
  title: z.string(),
  objective: z.string(),
  scope: z.array(z.string()),
  outOfScope: z.array(z.string()),
  acceptanceCriteria: z.array(z.string()),
  dependencies: z.array(z.string()),
  status: z.enum(WorkUnitStatuses),
  agentContextRefs: z.array(z.string()),
  suggestedFiles: z.array(z.string()).optional(),
  validationCommands: z.array(z.string()),
  createdAt: z.string(),
  updatedAt: z.string(),
});

export const DependencySchema = z.object({
  id: z.string(),
  fromWorkUnitId: z.string(),
  toWorkUnitId: z.string(),
  type: z.enum(["blocks", "requires", "relates_to"]),
  reason: z.string().nullable(),
});
```

13.11 Checkpoint and Runlog Schemas

```
export const CheckpointSchema = z.object({
  id: z.string(),
```

```

workUnitId: z.string(),
createdAt: z.string(),
summary: z.string(),
completed: z.array(z.string()),
notCompleted: z.array(z.string()),
filesChanged: z.array(z.string()),
issues: z.array(IssueSchema),
validationResult: z.enum(["passed", "failed", "partial", "not_run"]),
acceptanceCriteriaResult: z.enum(["passed", "failed", "partial", "not_checked"]),
nextRecommendation: z.string().nullable(),
});

export const RunlogEventSchema = z.object({
  id: z.string(),
  type: z.string(),
  timestamp: z.string(),
  actor: z.enum(["human", "aiqt", "agent", "system"]),
  summary: z.string(),
  relatedIds: z.array(z.string()),
  data: z.record(z.unknown()).optional(),
});

```

14. Initial File Shapes

14.1 Initial project.json

```

{
  "version": "0.5.0",
  "project": {
    "id": "PROJECT-001",
    "name": "current-folder-name",
    "objective": "",
    "targetUsers": [],
    "preferredAgent": null,
    "existingRepositoryPath": null,
    "createdAt": "ISO_DATE",
    "updatedAt": "ISO_DATE"
  },
  "context": {
    "constraints": [],
    "nonGoals": [],
    "technologyPreferences": [],
    "businessRules": [],
    "architectureNotes": []
  },
  "requirements": [],
  "decisions": [],
  "risks": [],
  "assumptions": [],
  "openQuestions": [],
  "quality": {
    "acceptanceCriteriaRequired": true,
    "validationRequiredBeforeDone": true,
    "preferredValidationCommands": []
  },
  "integrations": {}
}

```

14.2 Initial state.json

```

{
  "version": "0.5.0",
  "projectStatus": "draft",
  "currentMilestoneId": null,
  "currentWorkUnitId": null,
  "workGraph": {
    "milestones": [],
    "workUnits": [],
    "dependencies": []
  }
}

```

```

},
"checkpoints": [],
"lastAgentPacket": null,
"nextRecommendedCommand": "aiqt update",
"lastUpdatedAt": "ISO_DATE"
}

```

15. Workflow Engine

The workflow engine checks state, verifies command preconditions, enforces status transitions, selects next recommended actions, selects ready work units, applies state transitions, and appends runlog events. It must not contain command-line rendering logic.

```

export interface WorkflowEngine {
  canRun(action: WorkflowAction, context: WorkflowContext): GateResult;
  apply(action: WorkflowAction, input: unknown, context: WorkflowContext): Promise<WorkflowMutation>;
  getNextAction(context: WorkflowContext): NextActionResult;
  selectNextWorkUnit(context: WorkflowContext): WorkUnitSelectionResult;
}

```

16. Gate Rules

Command	Main Gate
aiqt update	Project initialized and canonical files validate.
aiqt plan	Project objective and enough context exist to generate at least one milestone and work unit.
aiqt next	Work graph exists, one ready work unit exists, and handoff gate passes.
aiqt checkpoint	A current work unit is in progress or a work unit is explicitly provided.
aiqt review	Project initialized and state validates.
aiqt export	Project initialized and requested export type is supported.
aiqt status	Canonical files can be read and schemas validate.

17. Runlog Health

- runlog.jsonl is append-only.
- Each line should be a valid JSON object matching RunlogEventSchema.
- Missing or unreadable runlog.jsonl is invalid state and exits with code 3.
- Malformed individual lines should not block aiqt status or aiqt next if project.json and state.json are valid.
- Malformed individual lines must be reported as warnings with severity medium and area runlog.
- JSON output should expose data.runlogHealth with totalLines, validLines, malformedLines, and malformedLineNumbers.

18. aiqt init Interaction Model

- Milestone 1 does not implement interactive prompts.

- aiqt init may accept --objective, --target-user, and --agent.
- If flags are omitted, objective remains empty, targetUsers remains empty, preferredAgent remains null, projectStatus remains draft, and nextRecommendedCommand remains aiqt update.
- Interactive project intake is implemented later through aiqt update.

19. Agent Packet Service

The agent packet service builds bounded context for one current work unit, includes relevant constraints only, injects acceptance criteria and out-of-scope boundaries, includes validation commands, and defines required agent output. It is out of scope for Milestone 1.

20. Testing Strategy

- Unit tests for schemas, IDs, version checks, transitions, and atomic writes.
- Integration tests for init/status/next/checkpoint/export workflows as they are introduced.
- Corrupted state fixtures for malformed project.json, malformed state.json, missing canonical files, future version files, and malformed runlog lines.
- JSON output contract tests to ensure stable keys and deterministic shape.

21. Key Design Decision

AIQT must behave as a workflow state engine, not a markdown document generator.

Architecture priority:
state -> graph -> packet -> checkpoint